NLP 220 Section 3

HW1 Overview, Practice 7

Homework 1

A. Feature Engineering for Naive Bayes & SVM
   a. Emphasis on **feature engineering** techniques & comparisons
   b. Update: sub-sampling training data for efficient SVM training
B. Sentiment Analysis on Stanford's total movie review corpus
   a. Emphasis on **algorithm** selection & comparisons

Part A: Feature Engineering for Naive Bayes & SVM

The dataset is a ~37k subset of the Amazon Book Reviews dataset

- **review/text** holds the content of the review ← **must be somehow included in 3 features**
- **review/score** holds a score (1-5 stars) ← **used to derive our labels**
- **Title** refers to the title of the book
- **review/summary** holds a summary or title of the review

1.1.3 Dataset Tasks & Deliverables

- "From this dataset, create a binary classification dataset"
  - review/score {1, 2, 3, 4, 5} → {0 : (1, 2), 1: (4, 5)}

- " Create a train/test split using the train_test_split" (90% train, 10% test)
  - The 10% test data is for <u>final evaluation</u>
  - The training dataset is quite **large**, **sub-sample it for debugging**
  - Good practice: you can further split it to have a <u>validation set</u> (just call train_test_split on your training set)
    - Use this to choose hyperparameters, test features, estimate test accuracy. Only evaluate on 10% test at end
    - Optional in this assignment, necessary for reliable research/model selection in practice

- "Print or plot the distribution of your classes/labels"
  - Printing is fine, but plots are nice and good practice generally.
  - No need to waste significant time trying to plot over print

Feature Engineering Deliverables

- You need 3 distinct features that process text (review/text and/or other text variables)
  - Can combine with non-text features like helpfulness, etc.
- ‘Distinct’: non-trivially different *to the model*
  - I.e. unigrams, bigrams, trigrams are very distinct from each other from a modeling perspective, even if similar to build as a feature engineer
  - Two features with/without stopwords filtering probably also distinct, though maybe a bit less
  - Unigrams differing only in vocab size, probably not distinct enough
  - **Whatever you have planned is probably fine,** but you can always ask me

Useful strategies

- Any time you're working with a lot of data, sub-sample it as you build your feature engineering/training pipeline
  - Minimize time spent waiting for your code to run when its not complete yet
- **Necessary* even for final SVM training**
  - svm.fit(...) training runs in between O(n_feat * n_obs^2) and O(n_feat * n_obs^3) time
  - In my benchmarking on nlp-gpu-01:
    - 5,000 features x 1,000 training points → trains in a few seconds
    - 5,000 features x 10,000 training points → trains in 5-8 minutes
    - 5,000 features x >20,000 training point → **did not complete in a reasonable time**

Handling SVM Training time

My suggestion:

- Implement your workflow AND choose hyper-parameters with a subset of the training set in the range of 1,000-2,000 points
  - Something that trains in a few seconds for you
- Run a final model training on 5,000-10,000 points (still requires sub-sampling)
  - Something that trains in a couple minutes for you
- Don’t run on all 20k+

<u>Tentative</u>: you won’t be graded on some minimum number of training points used, just report how many you do use that trains in a reasonable amount of time

Part B: Sentiment Analysis on Stanford's total movie review corpus

1. Use ngram for feature extraction (n, whether to use stopwords, other design decisions up to you)

2. The training dataset contains **25,000 total examples*** and is balanced. Further split this using train_test_split to create a training split (90%) and a held out validation split (10%), with a fixed random seed for reproducibility.

3. Briefly describe the models you used, and the ngram features you created.

4. Use 5 models to classify sentiment, and report accuracy on the validation split for each. You can use this validation split to help choose hyperparameters for each of the 5 models (you only need to report accuracy of best of each of the 5 model types).

5. Report the test set accuracy of the best of your 5 models.

*same training size issues may result! For algorithms that do not train in a reasonable amount of time on the full set, sub-sample and report the number of training points actually used.

General report writing tips:

- Does not need to be long, or explain things not asked for, though you won't be penalized for length unless its exceptionally hard to find answers.

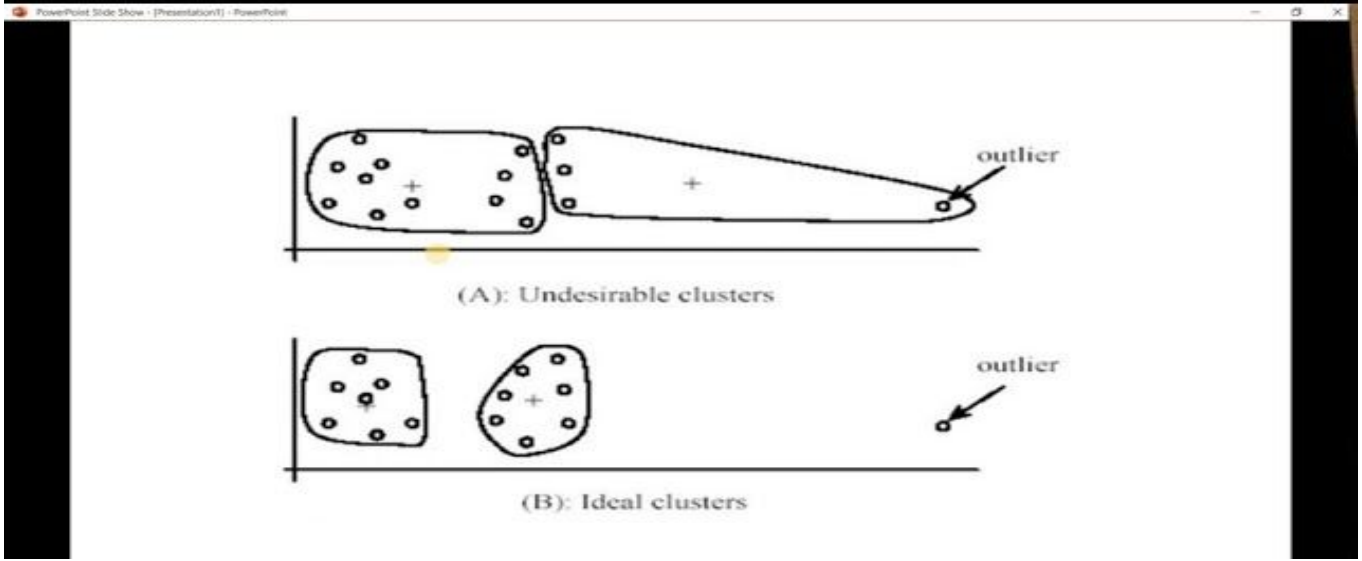
- Make it easy to skim:
  - Clear headings, tables for tabular data, etc.
  - Most people skim a research paper before deciding if it is worth reading more carefully
  - Report does not need to be paper length or quality, and Latex is optional: only use it if you think it helps you

Practice 7

The K-means algorithm is sensitive to outliers (True/False)

Practice 7

The K-means algorithm is sensitive to outliers (**True**/False)

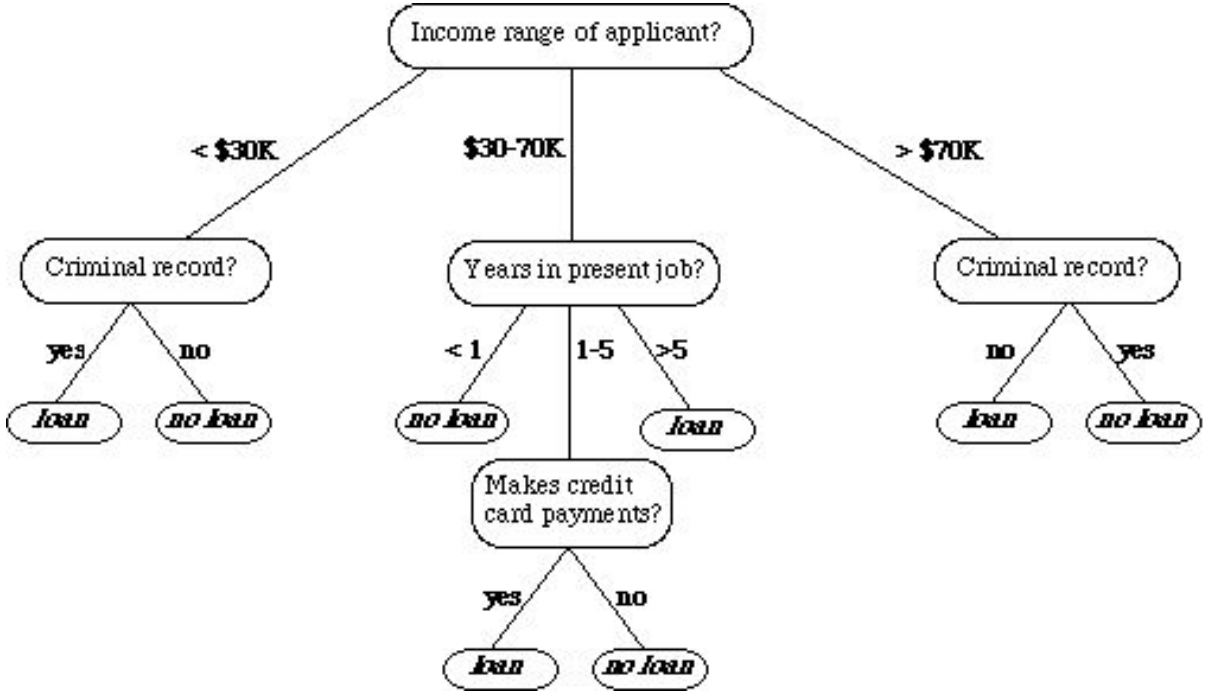


Practice 7

Decision trees can work with both numerical and categorical value (True/False)

Practice 7

Decision trees can work with both numerical and categorical value (**True**/False)



Practice 7

Decision trees have high variance (True/False)

Practice 7

Decision trees have high variance (**True**/False)

Why?

- Very few assumptions about the function you're learning
- Each leaf splits training data in (at best) half: training data at each split becomes smaller and smaller. Small within these lower nodes can lead to different predictions

Practice 7

Decision trees are generally unstable considering that a small change in the data set can result in a very different set of splits (True/False)

Practice 7

Decision trees are generally unstable considering that a small change in the data set can result in a very different set of splits (**True**/False)

Practice 7

The partitions in a classification are pure if the entropy is high (True/False)

Practice 7

The partitions in a classification are pure if the entropy is high (True/**False**)

-> The partitions in a classification are pure if the entropy is **zero**

Practice 7

Nearest Neighbor has classification runtime of O(LogN) (True/False)

Practice 7

Nearest Neighbor has classification runtime of O(LogN) (True/**False**)

Training: O(1) → do nothing

Inference:

Brute force: have to check distances from all N → O(N)

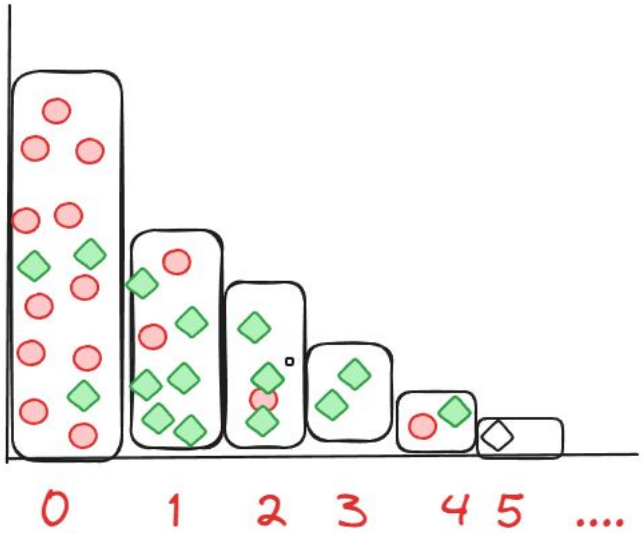
There are optimizations: see KDTree

Drawings from discussion

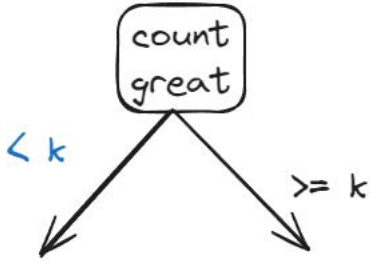
○ = negative

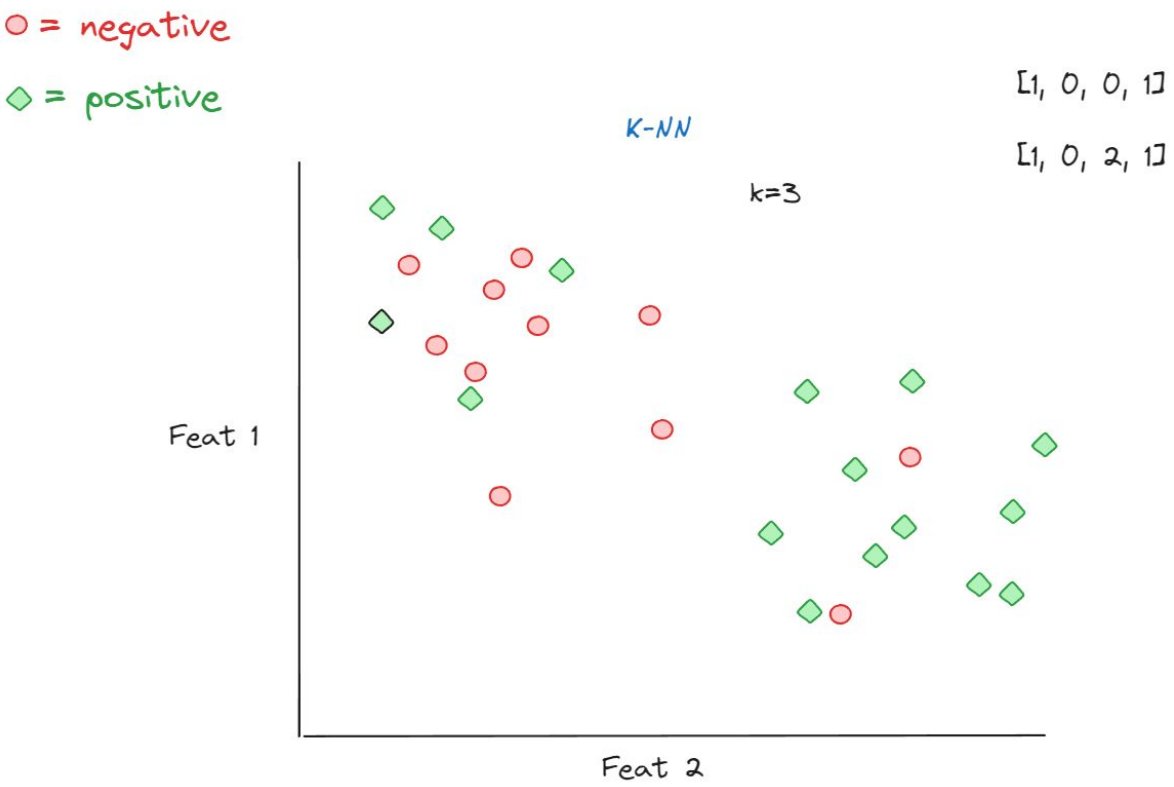
◇ = positive

@ decision tree node



# of times 'great' appears in review



Choose the k which maximizes information gain

Drawings from discussion